

Analisi e Classificazione del Dataset Forest Cover Types

Università degli Studi di Cagliari — progetto universitario rielaborato

Samuele Nonnis

1 — Introduzione

Il report confronta sedici modelli di classificazione sul dataset Forest Cover Types di scikit-learn. L'obiettivo è prevedere il tipo di copertura forestale, e quindi la specie arborea dominante, per celle di terreno di 30×30 m situate nella Roosevelt National Forest, in Colorado, a partire da feature cartografiche: quota, pendenza, distanze da acqua e strade, area selvaggia e tipo di suolo. Sono utilizzate tutte le 581.012 osservazioni, senza sottocampionamento.

Le sette classi sono fortemente sbilanciate, con la più numerosa che raccoglie il 48,76% delle osservazioni e la più rara lo 0,47%. Con questo sbilanciamento le metriche aggregate non bastano da sole. AdaBoost raggiunge accuracy 0,6688 e AUC-macro 0,8804 mentre la sua recall sulle classi 4, 5, 6 e 7 è 0,00 predicendo soltanto le tre classi più frequenti. I modelli sono perciò ordinati per F1 pesato con l'F1 macro riportato accanto, e ogni modello è documentato classe per classe.

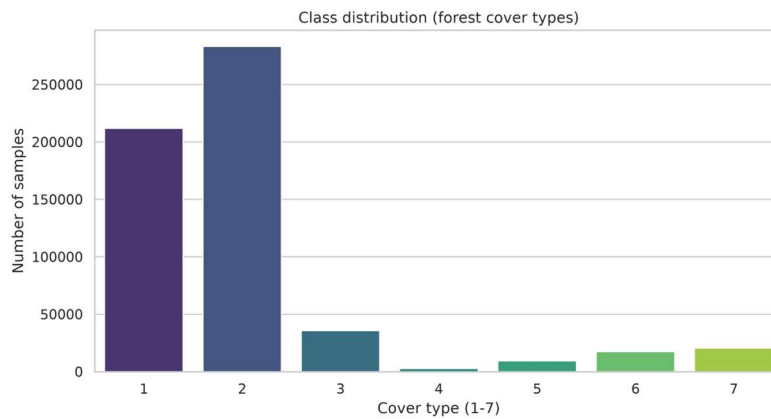
Il protocollo è identico per tutti e sedici i modelli. Gli iperparametri sono cercati su un subset stratificato di 100.000 osservazioni, il miglior estimatore viene riaddestrato sull'intero training set di 464.809 osservazioni e ogni modello è valutato una sola volta sullo stesso test set di 116.203 osservazioni, tenuto da parte. Lo sviluppo è stato eseguito in locale e l'addestramento finale su cloud, per contenere i tempi di calcolo.

2 — Esplorazione del Dataset

2.1 Descrizione del Dataset

Il dataset contiene 581.012 osservazioni e 7 classi che rappresentano le diverse specie di alberi. La distribuzione delle classi è fortemente sbilanciata: le classi Spruce/Fir (1) e Lodgepole Pine (2) costituiscono insieme l'85,22% delle osservazioni, mentre il restante 14,78% si divide tra le altre 5 classi. Cottonwood/Willow (4) e Aspen (5) sono le più rare, con lo 0,47% e l'1,63% del totale. Per via di questo sbilanciamento si utilizza l'F1-score pesato come metrica di riferimento.

Classe	Nome	Osservazioni	%
1	Spruce/Fir	211.840	36,46%
2	Lodgepole Pine	283.301	48,76%
3	Ponderosa Pine	35.754	6,15%
4	Cottonwood/Willow	2.747	0,47%
5	Aspen	9.493	1,63%
6	Douglas Fir	17.367	2,99%
7	Krummholz	20.510	3,53%



La classificazione si basa su 54 feature cartografiche:

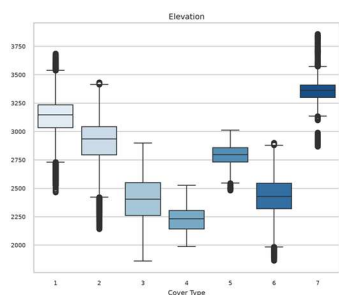
- 10 quantitative;
- 4 binarie che rappresentano l'area wilderness codificate in One-Hot Encoding;
- 40 binarie che rappresentano i tipi di suolo codificati in One-Hot Encoding.

Prima dell'analisi è stato applicato il Reverse One-Hot Encoding per compattare le 44 colonne binarie in 2 variabili categoriche native: `Wilderness_Area` e `Soil_Type`. A queste si sono aggiunte 3 feature ingegnerizzate, portando il dataset a 15 feature.

Tipo	Feature	Descrizione
Quantitativa	Elevation	Quota sul livello del mare (m)
Quantitativa	Aspect	Orientamento (°)
Quantitativa	Slope	Pendenza (°)
Quantitativa	Horiz. Dist. to Hydrology	Distanza orizzontale da corsi d'acqua (m)
Quantitativa	Vert. Dist. to Hydrology	Distanza verticale da corsi d'acqua (m)
Quantitativa	Horiz. Dist. to Roadways	Distanza dalle strade (m)
Quantitative (×3)	Hillshade 9am / Noon / 3pm	Indici di ombreggiatura in tre momenti (0–255)
Quantitativa	Horiz. Dist. to Fire Points	Distanza da punti di innesco incendi (m)
Ingegnerizzata	Distance_To_Hydrology	Distanza euclidea da corsi d'acqua
Ingegnerizzate (×2)	Elev_minus_VDH / Elev_plus_VDH	Combinano quota e distanza verticale idrica
Categorica	Wilderness_Area	Area naturale protetta (4 categorie)
Categorica	Soil_Type	Tipo di suolo (40 categorie)

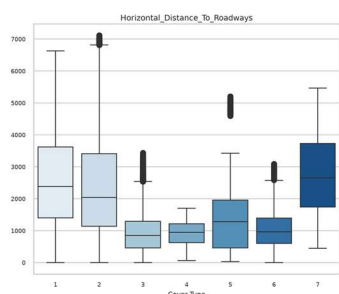
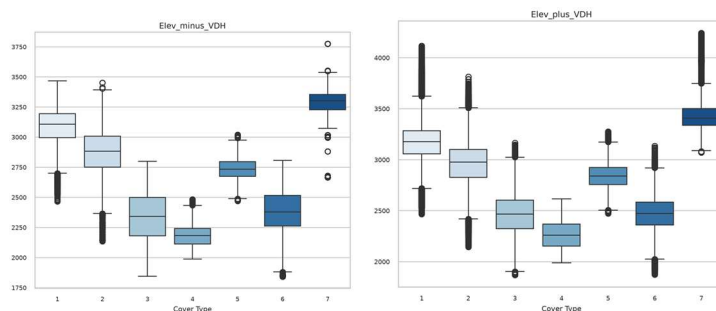
2.2 Analisi delle Feature Quantitative

Il dataset ora include 13 feature quantitative, analizzate tramite boxplot e ordinate per potere discriminativo rispetto alle 7 classi.



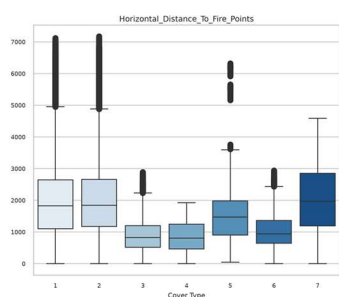
Elevation — quota in metri sul livello del mare L'altitudine determina temperatura, umidità e condizioni climatiche che definiscono l'habitat ideale per ciascuna specie. È utile per discriminare le specie adattate ai climi rigidi ad alta quota da quelle che si trovano nelle valli. È la feature più discriminativa, le distribuzioni per classe sono quasi tutte separate. La classe (7) occupa le quote più elevate (>3.300 m), con una distribuzione compatta e molti outlier verso quote superiori. La classe (1) si trova tra i 2.800–3.200 m, la (2) tra 2.500–3.000 m. Le classi 3, 4, 5, 6 occupano le zone più basse (<2.700 m) in maniera stratificata, con pochi o nessun outlier.

Elev_minus_VDH ed Elev_plus_VDH — feature ingegnerizzate Le due feature sono combinazioni lineari di Elevation e Vertical Distance to Hydrology, che cercano di catturare le interazioni tra quota e posizione rispetto ai corsi d'acqua. Le distribuzioni sono influenzate da Elevation, con lo stesso ordine e le stesse separazioni tra classi, ma la creazione di queste due feature può essere utile per i modelli ad albero. Durante la valutazione del modello tramite Permutation Importance, la feature Elev_minus_VDH è risultata quella più influente per il modello vincitore.



Horizontal Distance to Roadways — distanza dalle strade Indica quanto una zona è remota o accessibile: le specie di montagna, ad esempio, si trovano più lontane dalla rete stradale. Le classi 3, 4, 5, 6 hanno mediane che si collocano su distanze intorno ai 1.000 m, con distribuzioni compatte. Le classi 2 e 1 arrivano ai 2.000 m e 2.400 m, e la classe 7 arriva ai 2.700 m; queste tre classi hanno invece distribuzioni ampie. La feature è discriminativa, specialmente per isolare le classi 1, 2 e 7. Diverse classi hanno outlier che si posizionano su distanze

elevate.

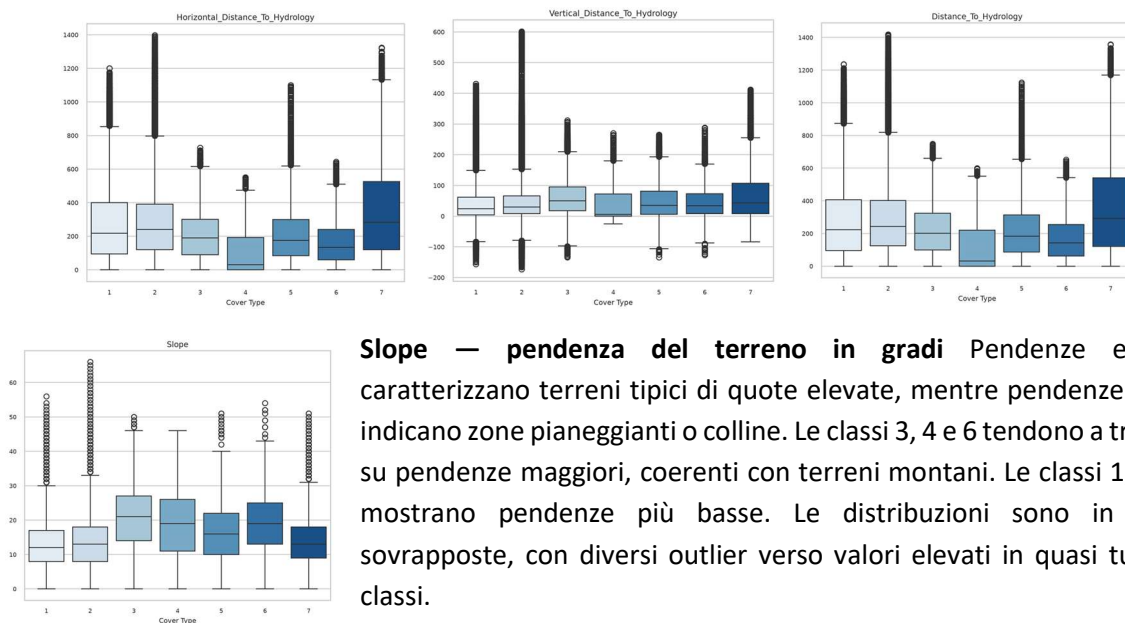


Horizontal Distance to Fire Points — distanza da punti di innesco incendi Misura la distanza storica da zone colpite da incendi. Alcune specie sono più resistenti al fuoco, al contrario di altre. Nessuna delle distribuzioni supera i 3.000 m, esclusi gli outlier. La (1) e la (2) mostrano distribuzioni ampie e quasi identiche. Le classi 3, 4, 5, 6 tendono a distanze inferiori e hanno distribuzioni compatte. La (7) non possiede outlier. Si osserva quindi che le specie di alta quota sono di solito più distanti da zone a rischio incendio, mentre altre crescono vicino ad aree

più vulnerabili al fuoco.

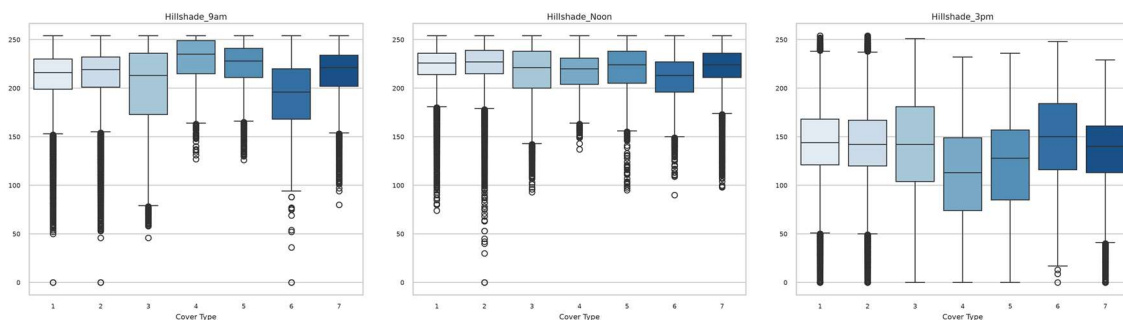
Horizontal / Vertical Distance to Hydrology e Distance to Hydrology — distanze da corsi d'acqua

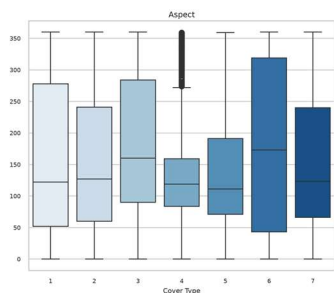
Distanza orizzontale, verticale ed euclidea dai corsi d'acqua, utilizzate per identificare le specie che necessitano di vivere in prossimità dell'acqua. La classe (4) è la più distinta in tutti e 3 i grafici, con distanze che hanno mediane intorno a 0 m, molto basse rispetto alle altre classi. La distanza verticale presenta valori bassi, che non superano i 50 m, e distribuzioni compatte e sovrapposte con numerosi outlier per tutte le classi. Per le distanze euclidea e orizzontale si notano distribuzioni più variegate.



Slope — pendenza del terreno in gradi Pendenze elevate caratterizzano terreni tipici di quote elevate, mentre pendenze basse indicano zone pianeggianti o colline. Le classi 3, 4 e 6 tendono a trovarsi su pendenze maggiori, coerenti con terreni montani. Le classi 1, 2 e 7 mostrano pendenze più basse. Le distribuzioni sono in parte sovrapposte, con diversi outlier verso valori elevati in quasi tutte le classi.

Hillshade 9am / Noon / 3pm Le tre feature simulano l'irraggiamento solare in tre momenti della giornata (0–255) e fungono da indicatori dell'esposizione al sole, che potrebbe influenzare la crescita di specie con diverse tolleranze alla luce. Le distribuzioni sono molto sovrapposte. Per 9am variano in un range di 170–250, con molti outlier verso il basso, e solo la classe (6) sembra discostarsi dalle altre. Per noon si hanno valori molto alti, distribuzioni compatte quasi uguali e outlier verso il basso. Per 3pm vi è un range di 70–180, con outlier in entrambe le direzioni. Complessivamente queste feature hanno un potere discriminativo minore rispetto alle altre variabili topografiche.





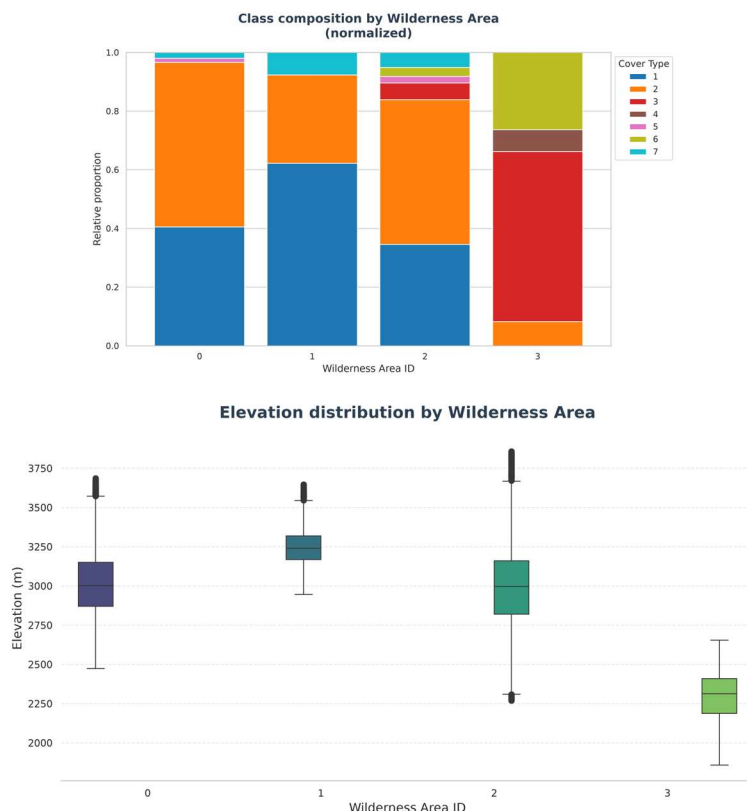
Aspect — orientamento del versante in gradi 0–360° Indica l'esposizione al sole e distingue se una specie predilige aree più ombrose o più soleggiate. Le distribuzioni sono sovrapposte tra la maggior parte delle classi, quindi la feature non ha una grande forza discriminativa. Le mediane sono comprese tra 110° e 160°, tranne per la classe (6), che mostra un valore più elevato (~175°) con una distribuzione più ampia rispetto alle altre; si presume quindi la presenza di questa specie su versanti esposti a nord-ovest o nord.

Inoltre solo la classe 4 presenta outlier verso valori alti. La feature non è un predittore importante, ma può essere utile per la classe (6).

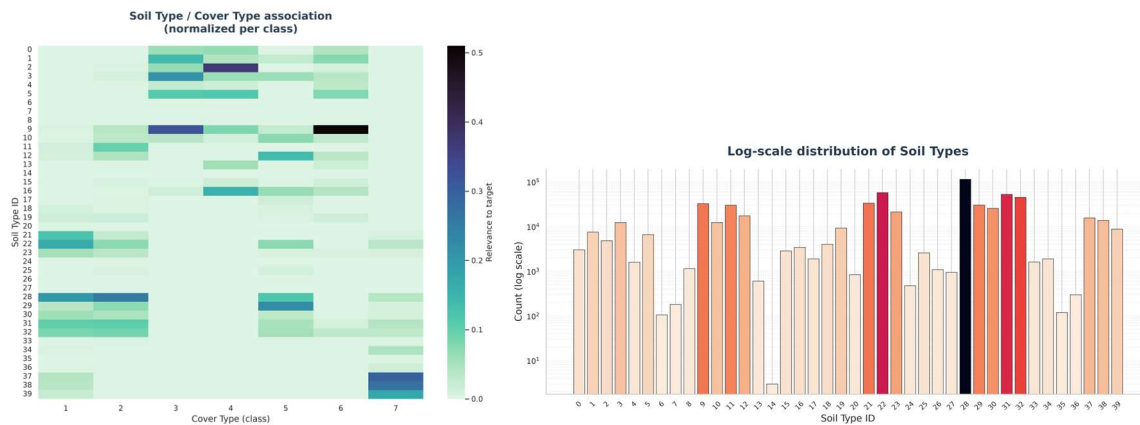
Sintesi. Elevation è la feature più discriminativa, assieme alle feature ingegnerizzate che le sono correlate. A seguire, Horizontal_Distance_To_Roadways e Horizontal_Distance_To_Fire_Points danno il loro contributo, e le distanze idriche sono utili per isolare la classe (4). Hillshade e Aspect sono le meno informative da sole, ma possono essere utili in combinazione con altre feature.

2.3 Analisi delle Feature Categoricalhe

Wilderness_Area — area naturale protetta Indica la zona di appartenenza della specie arborea, tra quattro aree protette. Questa feature è un buon predittore, soprattutto se utilizzata con l'altitudine. Dal grafico della composizione delle classi, le aree 0, 1 e 2 sono dominate dalle classi (1) e (2). L'area 3 si trova a quote più basse, come mostra il boxplot della distribuzione dell'altitudine. Inoltre l'area 3 è l'habitat principale delle classi (3) e (6) e ospita quasi la totalità della classe (4), che è quasi assente nelle altre aree.



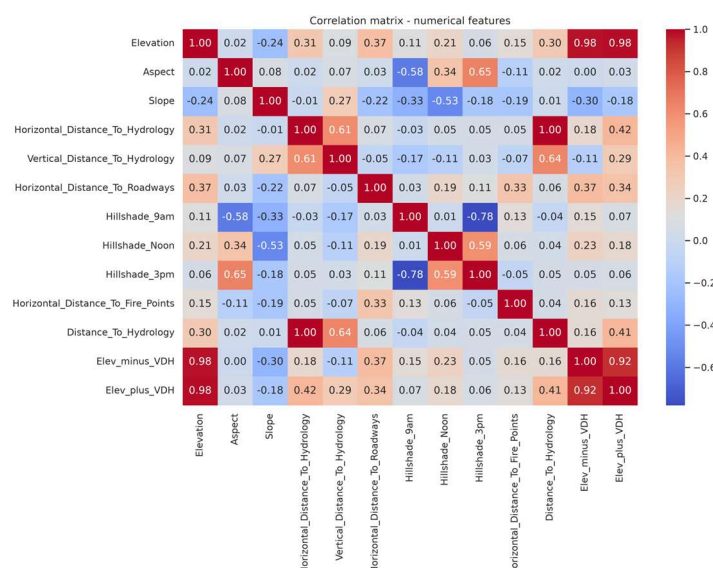
Soil_Type — tipo di suolo Il grafico della distribuzione logaritmica mostra un forte sbilanciamento: il Soil Type 28 è quello più frequente nelle osservazioni, mentre altri sono quasi assenti (come l'ID 14). La feature è molto discriminativa se si osservano le associazioni tra certe specie e certe tipologie di suolo. Dalla heatmap si nota che la classe (7) mostra un'associazione molto forte con i suoli 38 e 39, mentre la classe (6) è fortemente associata al suolo con ID 9. Le classi (1) e (2) sono distribuite su un range più ampio di suoli (21–32).



2.4 Matrice di Correlazione

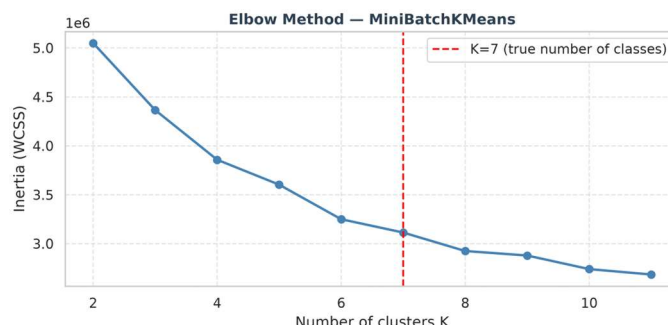
Correlazioni più rilevanti:

- **Elevation ↔ Elev_minus_VDH / Elev_plus_VDH ($r \approx 0,98$):** le due feature ingegnerizzate derivano dalla quota, pertanto questa correlazione era attesa.
- **Hillshade_9am ↔ Hillshade_3pm ($r = -0,78$):** queste due feature sono fortemente correlate negativamente per via dell'asimmetria tra l'irraggiamento solare al mattino e quello del pomeriggio.
- **Horiz. Distance to Hydrology ↔ Distance_To_Hydrology ($r \approx 1,00$):** la feature ingegnerizzata è fortemente correlata con la componente orizzontale, poiché la componente verticale risulta molto piccola.
- **Aspect ↔ Hillshade_3pm ($r = 0,65$):** le due feature sono correlate positivamente, poiché l'orientamento del versante condiziona direttamente l'ombreggiatura.



2.5 Analisi di Clustering e PCA

A scopo esplorativo si è svolta un'analisi di clustering. Per via delle dimensioni del dataset, l'algoritmo K-Means classico sarebbe risultato computazionalmente oneroso. Per questo motivo si è scelto di utilizzare il MiniBatchKMeans, che riduce i tempi di calcolo aggiornando i centroidi su batch di osservazioni a ogni iterazione.

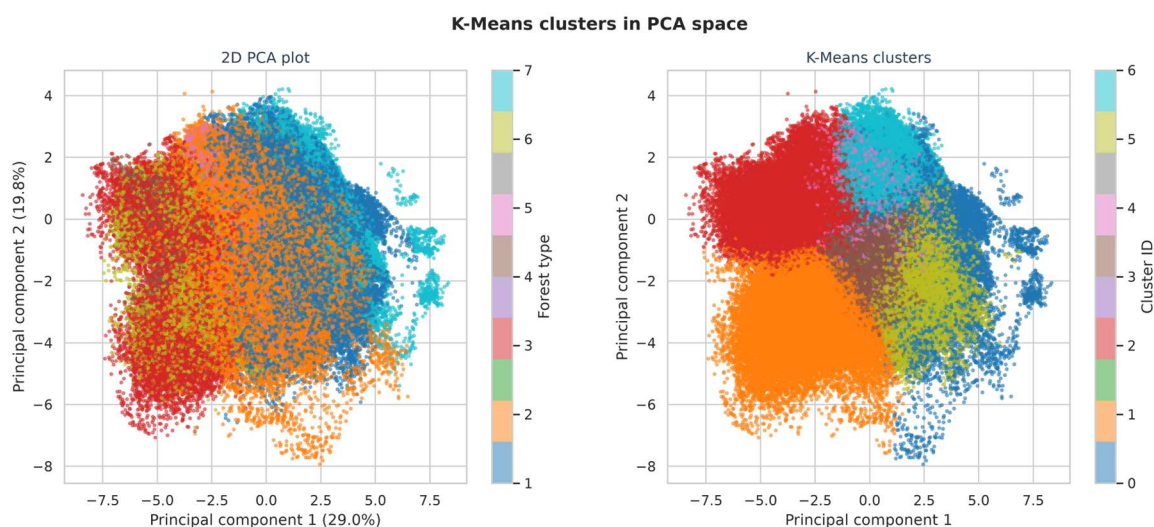


Per determinare il numero di cluster K è stata utilizzata la regola del gomito, monitorando la within-cluster variation al variare di K tra 2 e 11. Essendo l'analisi solamente esplorativa, si è scelto di fissare K=7, anche perché è il numero di specie forestali da prevedere. I cluster ottenuti sono stati valutati tramite due metriche:

- **Adjusted Rand Index (ARI) = 0,0476:** I cluster non coincidono con le classi, le specie non formano regioni compatte nello spazio delle feature, ma zone complesse e intersecate.
- **Silhouette Score = 0,1621:** Cluster poco separati e parzialmente sovrapposti, coerentemente con la difficoltà di separare le classi.

Per confrontare la distribuzione reale delle classi con i raggruppamenti creati dall'algoritmo K-Means si è svolta un'analisi dei pesi della PCA. La prima componente cattura il 29% della varianza delle feature quantitative ed è composta dalle tre feature di quota ($Elev_plus_VDH \approx 0,48$, $Elevation \approx 0,47$, $Elev_minus_VDH \approx 0,43$). La seconda aggiunge un altro 19,8%.

Nonostante ciò, le classi reali si distribuiscono lungo l'asse di PC1 con molte sovrapposizioni. Questo spiega il basso ARI, dovuto al fatto che l'altitudine ha il segnale più forte, ma da sola è insufficiente. Per separare correttamente le regioni è quindi necessario utilizzare modelli non lineari.



3 — Preprocessing

3.1 Split del Dataset

Per prevenire data leakage e conservare le proporzioni delle classi, il dataset è stato suddiviso in modo stratificato in train-set (80%) e test-set (20%), ottenendo 464.809 e 116.203 osservazioni. Per ridurre la complessità computazionale e i tempi di calcolo, la ricerca degli iperparametri si è svolta utilizzando un subset di tuning (`X_train_tune`) ridotto e stratificato a 100.000 campioni. I migliori iperparametri vengono poi utilizzati per l'addestramento di ciascun modello sull'intero train-set di 464.809 osservazioni. Per l'SVM, a causa della complessità computazionale $O(n^2)$, si è deciso di limitare l'addestramento a un subset di 100.000 campioni (`X_train_svm`).

3.2 Scaling delle Feature

Per le trasformazioni delle feature l'approccio è stato diverso in base alla famiglia di modelli, tramite `ColumnTransformer`:

- **Modelli lineari e basati su distanze (LR, KNN, SVM, MLP, LDA/QDA):** essendo sensibili alla scala delle feature, le sole 13 feature quantitative sono state normalizzate utilizzando `StandardScaler`, mentre quelle categoriche sono state codificate con `OneHotEncoder`.
- **Modelli basati su alberi:** questi modelli sono basati su confronti e sono quindi immuni a eventuali problemi di scala. Hanno quindi ricevuto le feature direttamente senza trasformazioni.

3.3 Gestione dello Sbilanciamento delle Classi

Per gestire lo sbilanciamento sono stati usati due approcci:

- **`class_weight`:** testando diversi pesi come iperparametri sui modelli Random Forest e LightGBM.
- **SMOTE:** utilizzato per bilanciare le classi per il modello MLP.

Gli altri modelli (Logistic Regression, SVM, Decision Tree, LDA/QDA, ecc.) sono stati addestrati senza bilanciamento. Come mostrano i risultati, il metodo più efficace sulle classi rare è stato l'addestramento sull'intero dataset, che ha portato la recall della classe 4 da 0,61 a 0,88 e quella della classe 5 da 0,43 a 0,88 nel modello vincitore.

3.4 Ottimizzazione della Memoria

Il dataset Coverttype è esteso e per gestire l'esecuzione e l'addestramento specialmente su cloud, sono state necessarie ottimizzazioni per ridurre l'utilizzo di memoria RAM:

- **Downcasting dei tipi di dato:** le feature continue allocate come `float64` sono state convertite in `float32`. Questo ha permesso di ridurre l'occupazione di memoria del dataset, evitando crash per via della saturazione della RAM o errori di disco.
- **Gestione dei residui:** al termine dell'esecuzione di ciascun modello, la funzione `evaluate_model` invoca il garbage collector di Python e chiude tutte le figure aperte da matplotlib.

4 — Scelte Tecniche

Per uniformare e automatizzare il confronto tra tutti i modelli e salvare i risultati sono state implementate le seguenti soluzioni:

- **Pipeline per la valutazione:** è stata creata una procedura `evaluate_model` che centralizza il calcolo delle metriche per ogni modello. La procedura calcola le metriche principali, genera il classification report e salva la matrice di confusione in formato PNG per ogni modello.
- **Logging:** per evitare la perdita di informazioni durante l'addestramento e tracciare i risultati, la funzione `print()` standard è stata sovrascritta da un logger, utilizzato per duplicare ogni output testuale sia su schermo sia su un file di log (`run_log.txt`).
- **Grafici:** ciascun grafico generato all'interno del notebook viene salvato automaticamente nella cartella `figures`.

5 — Metriche e Valutazione

La metrica principale è l'F1-score pesato, perché pondera ogni classe per la sua numerosità e riflette quindi le prestazioni su un test set che conserva lo sbilanciamento dei dati. Accanto a esso in ogni tabella, è riportato l'F1-score macro, che pesa allo stesso modo tutte e sette le classi e cala non appena un modello va male su quelle rare. La distanza tra i due misura quanta parte del punteggio di un modello poggia sulle classi maggioritarie. Viene riportata anche l'accuracy, per confronto con la letteratura.

Le metriche aggregate non mostrano su quali classi un modello abbia fallito, perciò per ciascun modello vengono generati anche un classification report con precision, recall ed F1 per ogni classe e una confusion matrix normalizzata per riga. Le stesse informazioni per classe sono raccolte in due heatmap che confrontano tutti i modelli classe per classe. L'AUC-macro OvR e le relative curve ROC sono calcolate per ogni modello da cui è possibile estrarre le `predict_proba`, mentre le curve precision-recall one-vs-rest sono prodotte per i cinque modelli migliori. Tutti i grafici sono salvati nella cartella `figures`. Le metriche sono calcolate dalla funzione `evaluate_model` sul test-set completo, per ogni modello riaddestrato sul training set con i suoi migliori iperparametri.

6 — Sviluppo e Addestramento dei Modelli

Sono stati addestrati 16 modelli di classificazione e per ciascuno la ricerca degli iperparametri è stata svolta con `HalvingGridSearchCV` — `HalvingRandomSearchCV` per `LightGBM` — sul subset di tuning ridotto e stratificato a 100.000 osservazioni (`X_train_tune`), utilizzando come metrica l'F1-weighted per via dello sbilanciamento del dataset. I migliori iperparametri sono stati poi impiegati per l'addestramento sull'intero train-set di 464.809 osservazioni. Ogni modello è incapsulato in una pipeline che applica il `ColumnTransformer` in base alla tipologia di modello. Alcuni modelli seguono invece un'altra pipeline per via di necessità specifiche: i modelli parametrici (LDA, QDA, Naive Bayes) non hanno iperparametri significativi e sono stati valutati direttamente con `cross_val_score` (`cv=5`); l'SVM viene addestrato su un subset dedicato di 100.000 campioni (`X_train_svm`) a causa della complessità $O(n^2)$; l'MLP utilizza nella pipeline SMOTE per gestire le classi rare.

Ogni modello addestrato è stato valutato sul test-set completo (116.203 osservazioni) con la funzione `evaluate_model`, che calcola le metriche principali, genera il classification report per classe e salva la confusion matrix. Le curve ROC (one-vs-rest) e l'AUC-macro sono state calcolate per tutti i modelli da cui è stato possibile estrarre le `predict_proba`, tranne l'SVM, avendo `probability=False` per contenere i tempi di addestramento.

6.1 Modelli Lineari e Parametrici

Tutti i modelli di questa famiglia utilizzano lo `scaler` `preprocessor_linear` e ricevono le 13 feature quantitative normalizzate con `StandardScaler` e le 2 categoriche con `OneHotEncoder`.

Logistic Regression (L1 vs L2): si è studiata una regressione logistica "classica", che utilizza il solver `saga` (`max_iter=500`), l'unico solver di `scikit-learn` che supporta entrambe le penalizzazioni L1, L2.

- $C \in \{0, 1, 10\}$ — forza di regolarizzazione;
- $penalty \in \{L1, L2\}$ — tipo di penalità.

Logistic Regression + PCA: variante della regressione logistica dove viene applicata la riduzione dimensionale. Prima del classificatore si applica una PCA con `n_components=0.95`, che conserva il 95% della varianza e seleziona automaticamente le prime 12 componenti. Come iperparametri si testa la regolarizzazione di default (L2) con $C \in \{0, 1, 10\}$. L'obiettivo era valutare se la PCA migliorasse la separabilità, ma i risultati mostrano invece una perdita informativa sulle classi rare.

Logistic Regression con splines: per dare al modello la capacità di catturare relazioni non lineari, le 13 feature quantitative passano dentro uno `SplineTransformer` cubico (`degree=3`). Gli iperparametri testati sono:

- $n_knots \in \{3, 4, 5, 6\}$ — regolano la flessibilità delle spline;
- $C \in \{0, 1, 10, 100\}$ — forza di regolarizzazione (L2 di default).

LDA: per via della semplicità del modello vengono utilizzati i parametri di default e `cross-validation`.

QDA: in questa variante è possibile testare la regolarizzazione con `reg_param=0.5`, che stabilizza l'inversione delle matrici di covarianza. Anche in questo caso si utilizza la `cross-validation` (`cv=5`).

Naive Bayes: il modello è semplice e ci si limita a osservarne il comportamento con i parametri di default. Il Naive Bayes assume indipendenza condizionale e distribuzioni gaussiane per ogni feature data la classe, ma questa ipotesi è fortemente violata in questo dataset, con feature geografiche anche molto correlate. Questo modello è probabilmente il meno adatto al dataset e infatti risulta essere quello con le prestazioni peggiori nel confronto. Anche in questo caso si esegue una `cross-validation`.

6.2 Modelli Basati su Distanze e Margini

KNN: viene testato il miglior numero di vicini, con $n_neighbors \in \{1, 3, 4, 5, 6, 7, 11\}$. La metrica utilizzata è quella di default di `scikit-learn`, che è la distanza euclidea, calcolata sull'output di `preprocessor_linear`. Grazie alla standardizzazione è possibile utilizzarla perché sulle feature grezze la quota dominerebbe una pendenza misurata in gradi. Non sono state testate metriche alternative e la codifica one-hot lascia due tipi di suolo diversi sempre alla stessa distanza fra loro.

SVM con kernel RBF: il modello è stato addestrato sul subset dedicato `X_train_svm` di 100.000 campioni per via della complessità $O(n^2)$, sul quale vengono svolti sia la ricerca degli iperparametri sia il refit. Si è reso necessario porre il parametro `probability=False` e disattivare la stima delle probabilità per contenere i tempi di calcolo; di conseguenza non è stato possibile calcolare la curva ROC. Si è scelto `kernel='rbf'` per via della natura non lineare del dataset, che permette di creare confini decisionali radiali. Gli iperparametri testati sono:

- `C` ∈ {1, 10} — regola la tolleranza dei punti misclassificati;
- `gamma` ∈ {scale, auto} — regola l'ampiezza del kernel.

6.3 Alberi di Decisione ed Ensemble

I modelli di questa sezione ricevono le feature senza scaling tramite `preprocessor_trees`, ad eccezione di Decision Tree, Bagging e AdaBoost che usano `preprocessor_linear` perché le loro implementazioni scikit-learn non gestiscono feature categoriche native.

Decision Tree (pruned): albero decisionale singolo, ottimizzato tramite pruning per evitare l'overfitting. Il modello, a differenza degli altri alberi, riceve lo scaler `preprocessor_linear`, poiché sklearn non gestisce feature categoriche su alberi classici. Combinando il limite di profondità e il pruning si ottiene un albero singolo abbastanza accurato. Gli iperparametri testati sono:

- `max_depth` ∈ {10, 20, None} — profondità massima dell'albero;
- `criterion` ∈ {gini, entropy} — metrica d'impurità per lo split;
- `ccp_alpha` ∈ {0.0, 0.0001, 0.001} — aggressività del cost-complexity pruning.

Random Forest: dato lo sbilanciamento del dataset, per questo modello si sono sviluppati due dizionari che partono dai pesi bilanciati e li moltiplicano per 2 e per 3 sulle classi rare.

- `n_estimators` ∈ {50, 100, 150} — numero massimo di alberi nell'ensemble;
- `max_depth` ∈ {3, 5, 7, None} — profondità massima dei singoli alberi;
- `class_weight` ∈ {balanced, balanced_subsample, dict_custom_x2, dict_custom_x3}.

Bagging Classifier: anche il bagging di default non supporta classi categoriche, pertanto utilizza lo scaler `preprocessor_linear`. Gli iperparametri testati sono:

- `n_estimators` ∈ {50, 100, 150} — numero massimo di alberi nell'ensemble;
- `max_samples` ∈ {0,5, 0,8, 1,0} — percentuale di campioni bootstrappati per ogni albero.

Gradient Boosting: gli iperparametri testati sono:

- `learning_rate` ∈ {0,01, 0,1} — learning rate;
- `n_estimators` ∈ {50, 100, 150} — numero massimo di alberi nell'ensemble;
- `max_depth` ∈ {3, 5, 7} — profondità massima dei singoli alberi.

XGBoost: si incapsula `XGBClassifier` in una classe `CustomXGB` che utilizza `LabelEncoder` per mappare le etichette 1–7 delle classi. Il modello è addestrato con `tree_method='hist'`, un algoritmo eseguito su CPU per evitare errori di memoria su GPU. Per un confronto diretto si testano gli stessi parametri del Gradient Boosting.

LightGBM: per questo modello è possibile testare molti iperparametri, però è stato necessario bilanciare anche i tempi di calcolo; per questa ragione viene utilizzato `HalvingRandomSearchCV`, testando questi iperparametri:

- `learning_rate` ~ $U(0,01, 0,21)$ — contributo di ogni nuovo albero;
- `n_estimators` ~ `randint(50, 200)` — alberi nell'ensemble;
- `max_depth` ~ `randint(3, 10)` — profondità massima del singolo albero;
- `min_child_samples` ~ `randint(10, 100)` — regolarizzazione sulle foglie;
- `subsample` ~ $U(0,5, 0,9)$ — percentuale di righe campionate per ogni albero;
- `colsample_bytree` ~ $U(0,5, 0,9)$ — percentuale di feature usate per ogni albero;
- `class_weight` ∈ {balanced, None, dict_custom_x2} — strategie di penalizzazione.

AdaBoost: sono stati testati i seguenti iperparametri:

- `n_estimators` ∈ {50, 100, 150} — numero di stump;
- `learning_rate` ∈ {0,1, 0,5, 1,0} — contributo di ogni stump.

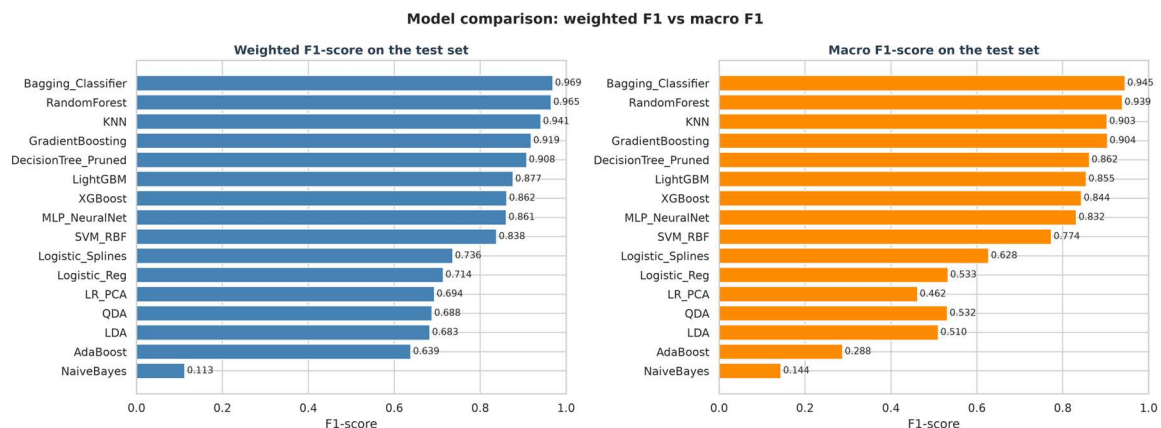
6.4 Rete Neurale

`MLPClassifier` utilizza l'optimizer adam con `max_iter=1000` ed `early_stopping=True`. Per le classi rare viene utilizzata `ImbPipeline` per applicare SMOTE. Si è anche implementata una variante `AdaptiveSMOTE` che riduce `k_neighbors` quando la classe più piccola ha meno di 6 campioni, evitando crash. Questo si è rivelato necessario durante lo sviluppo in locale, poiché non era possibile utilizzare tutto il dataset. Per contenere i tempi, gli iperparametri vengono selezionati su 50.000 osservazioni:

- `hidden_layer_sizes` ∈ {(64,), (32, 16), (64, 32, 16)} — architetture dei layer nascosti;
- `alpha` ∈ {0.0001, 0.05} — peso della regolarizzazione L2.

7 — Confronto dei Modelli

Tutti i 16 modelli sono stati addestrati, ottimizzati e valutati sullo stesso test-set contenente 116.203 osservazioni. La tabella che segue riassume i risultati ordinati per F1-score sul test set, in ordine decrescente, e nella figura vengono messi a confronto l'F1 pesato e l'F1 macro. Vengono anche mostrati i valori di Accuracy ed AUC-macro raggiunti da ciascun modello.



Modello	F1 CV	F1 Test	F1 Macro	Accuracy	AUC-macro
Bagging Classifier	0.9190	0.9693	0.9452	0.9693	0.9986
Random Forest	0.9028	0.9654	0.9389	0.9655	0.9988
KNN	0.8911	0.9413	0.9032	0.9413	0.9440
Gradient Boosting	0.8864	0.9187	0.9040	0.9189	0.9908
Decision Tree Pruned	0.8546	0.9083	0.8620	0.9092	0.9578
LightGBM	0.8701	0.8766	0.8548	0.8749	0.9890
XGBoost	0.8482	0.8622	0.8437	0.8630	0.9854
MLP NeuralNet	0.8213	0.8610	0.8318	0.8594	0.9851
SVM RBF	0.8328	0.8383	0.7736	0.8406	N/A
Logistic Splines	0.7378	0.7364	0.6278	0.7415	0.9483
Logistic Reg	0.7181	0.7142	0.5330	0.7237	0.9362
LR + PCA	0.6970	0.6937	0.4620	0.7062	0.9244
QDA	0.6874	0.6882	0.5317	0.6905	0.9237
LDA	0.6837	0.6830	0.5104	0.6800	0.9019
AdaBoost	0.6425	0.6388	0.2876	0.6688	0.8804
Naive Bayes	0.1121	0.1127	0.1441	0.1222	0.8112

7.1 Lettura dei Risultati per Famiglia

Il confronto mostra che vi è una gerarchia tra le famiglie di modelli.

Gli ensemble di alberi basati su bagging (Bagging Classifier e Random Forest) occupano le prime due posizioni con $F1 \approx 0,97$, seguiti dal KNN (0,941) e dai modelli di boosting (Gradient Boosting 0,919, LightGBM 0,877, XGBoost 0,862). Il singolo albero potato raggiunge lo 0,908 mentre l'MLP (0,861) e l'SVM RBF (0,838) si collocano nella fascia intermedia.

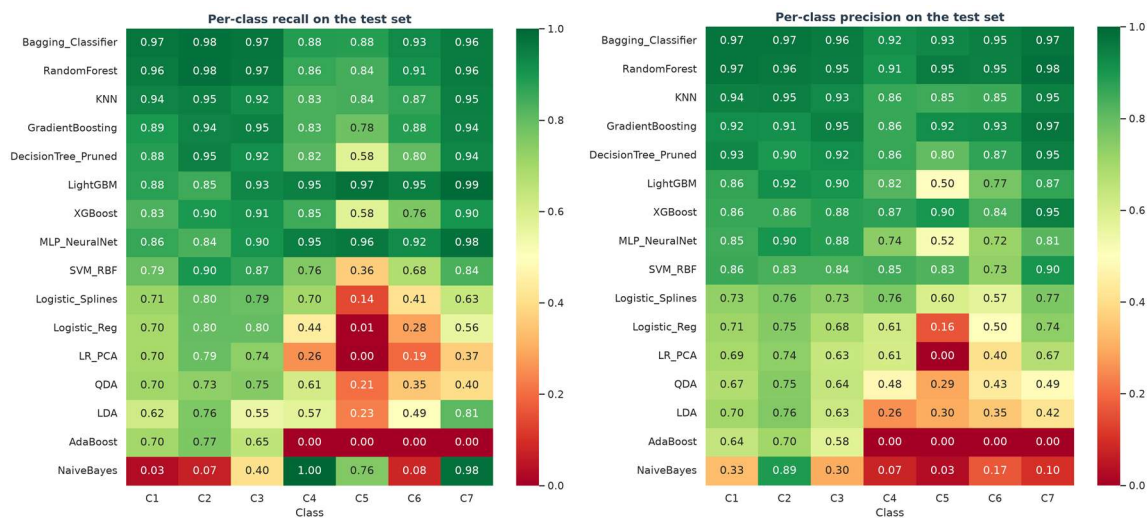
I modelli lineari e parametrici (Logistic Regression e varianti, LDA, QDA) si fermano invece tra 0,68 e 0,74, mentre AdaBoost (0,639) e soprattutto Naive Bayes (0,113) stanno alla fine della classifica.

Il dataset contiene variabili geografiche continue come quota, pendenza e distanze, che generano confini decisionali non lineari. I modelli lineari quindi non possono accedere a una parte di informazione che nemmeno la grande quantità di dati riesce a fornire, mentre i modelli spaziali e gli ensemble di alberi possono superare questo limite.

L'ordinamento dipende anche da quale aggregato si legge. Random Forest ha l'AUC-macro più alta di tutti e sedici i modelli (0,9988, sopra lo 0,9986 del vincitore) pur essendo seconda per F1 pesato, e il Gradient Boosting sta sotto al KNN per F1 pesato (0,9187 contro 0,9413) ma sopra per F1 macro (0,9040 contro 0,9032).

7.2 Comportamento per Classe tra i Modelli

Le due heatmap seguenti riportano recall e precision per classe di tutti i 16 modelli, con le righe nell'ordine della classifica.



Si osservano tre comportamenti:

- AdaBoost raggiunge accuracy 0,6688 e AUC-macro 0,8804, mentre la sua recall sulle classi 4, 5, 6 e 7 è 0,00, questo dimostra che il modello predice le sole tre classi più frequenti.
- LR + PCA mostra lo stesso comportamento sulla classe 5 (recall 0,00) e la Logistic Regression altrettanto (0,01).
- Il Naive Bayes raggiunge recall 1,00 sulla classe 4 con precision 0,07. Il modello assegna una porzione consistente del test set alla classe più rara, pertanto detiene l'F1 pesato più basso di tutti (0,1127) ma un F1 macro superiore (0,1441).

7.3 Il Contributo dei Dati

Il confronto con le prestazioni di una run su subset ridotto a 25.000 osservazioni, contro quelle ottenute sul dataset completo, mostra che mentre i modelli lineari non migliorano oltre un certo punto indipendentemente dalla quantità di dati (o addirittura peggiorano a causa del rumore), i modelli spaziali e gli ensemble di alberi beneficiano della presenza di più dati. Anche se il miglioramento delle prestazioni era prevedibile dato il passaggio a un dataset di grandezza superiore, il diverso comportamento dei modelli conferma le conclusioni.

Modello	F1 25k	F1 Full (464k)	Delta
KNN	0.820	0.941	+0.121
Bagging	0.862	0.969	+0.107
Decision Tree	0.752	0.908	+0.156
Random Forest	0.844	0.965	+0.121
Logistic Splines	0.736	0.736	0.000
LDA	0.690	0.683	-0.007
AdaBoost	0.643	0.638	-0.005

8 — Modello Vincitore — Bagging Classifier

Il Bagging Classifier è risultato il modello più stabile e accurato su tutte le metriche principali. Di seguito vengono riassunte le prestazioni:

Metrica	Valore
F1-Score pesato (Test)	0.9693
F1-Score macro (Test)	0.9452
Accuracy (Test)	0.9693
AUC-macro OvR	0.9986
F1 Bootstrap medio	0.9692
IC Bootstrap 95%	[0.9682, 0.9702]
Dimensione del test set	116.203 istanze

Il modello è stato in grado di riconoscere in modo stabile anche le classi rare, che erano il punto più critico del progetto. Di seguito il classification report:

Classe	Precision	Recall	F1-Score	Support
1 — Spruce/Fir	0.97	0.97	0.97	42.368
2 — Lodgepole Pine	0.97	0.98	0.97	56.661
3 — Ponderosa Pine	0.96	0.97	0.96	7.151
4 — Cottonwood/Willow	0.92	0.88	0.90	549
5 — Aspen	0.93	0.88	0.90	1.899
6 — Douglas Fir	0.95	0.93	0.94	3.473
7 — Krummholz	0.97	0.96	0.97	4.102
Media pesata	0.97	0.97	0.97	116.203

Le classi dominanti 1 e 2 raggiungono $F1 = 0,97$, ma anche le classi rare 4 e 5, nonostante la rarità, riescono a essere classificate in modo efficace.

Per escludere la presenza di varianza dovuta a split fortunati si è eseguita una validazione bootstrap sul test set. Sono stati estratti 1.000 campioni con reinserimento e calcolato l'F1-score su ciascuno, producendo un intervallo di confidenza al 95%. Si è ottenuto un F1 medio di 0,9692 e un intervallo di confidenza pari a [0,9682, 0,9702].

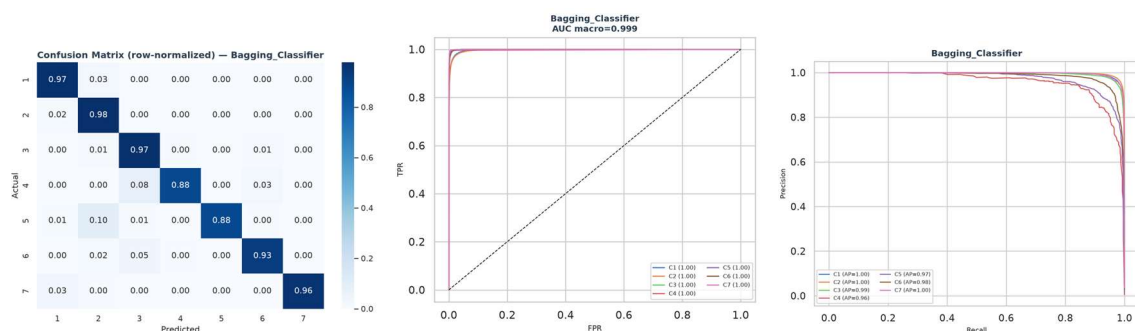
9 — Dettaglio per Modello

Per ciascuno dei 16 modelli vengono affiancate la confusion matrix e la curva ROC One-vs-Rest, con una curva per ciascuna delle 7 classi, e per i cinque modelli di testa anche le curve precision-recall con l'Average Precision di ogni classe. Le matrici di confusione sono normalizzate per riga, poiché con i support originali, i conteggi grezzi renderebbero illeggibili gli errori sulle classi rare. I risultati sono ordinati per F1-score sul test-set e vengono analizzati. Si ricorda che la curva ROC dell'SVM RBF non è disponibile per motivi di efficienza computazionale.

Con classi così sbilanciate la curva ROC è una metrica poco affidabile. Tutti i 15 modelli per cui è stato possibile calcolarlo raggiungono un AUC-macro $\geq 0,8112$, AdaBoost compreso, pur non predicendo mai quattro delle sette classi. Per questa ragione i cinque modelli di testa riportano anche le curve precision-recall.

1 — Bagging Classifier

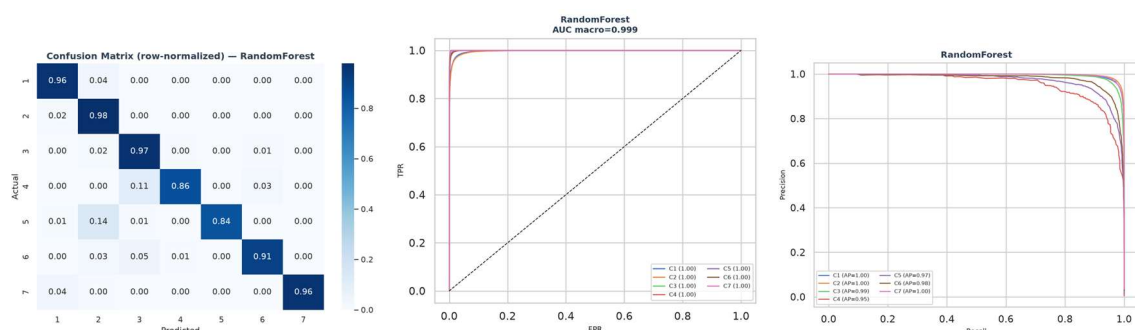
F1 CV = 0.919 | F1 Test = 0.969 | F1 Macro = 0.945 | Accuracy = 0.969 | AUC-macro = 0.999



La matrice è quasi diagonale, con errori che si concentrano quasi tutti sulla frontiera tra le classi 1 e 2. Le classi rare 4 e 5 hanno recall molto alte e tutte e 7 le curve ROC coincidono con AUC quasi 1,00. A differenza della Random Forest, il Bagging non introduce la selezione casuale delle feature, permettendo a ogni albero di utilizzare tutte le 15 feature. Si hanno quindi alberi singoli più potenti e più correlati tra loro, ma abbastanza diversi grazie al campionamento bootstrap.

2 — Random Forest

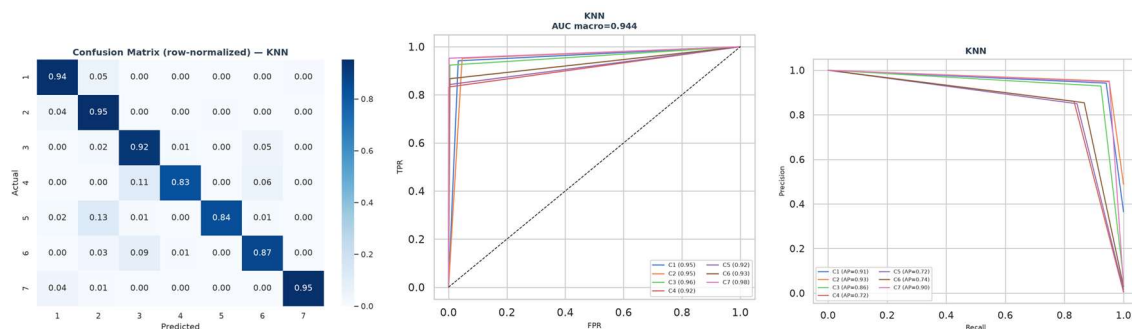
F1 CV = 0.903 | F1 Test = 0.965 | F1 Macro = 0.939 | Accuracy = 0.966 | AUC-macro = 0.999



Pattern quasi uguale al Bagging, con diagonale forte e poche confusioni. Le classi 4 e 5 hanno recall un po' inferiori al Bagging, probabilmente a causa della selezione casuale delle feature. AUC-macro = 0,999 come nel Bagging.

3 — KNN

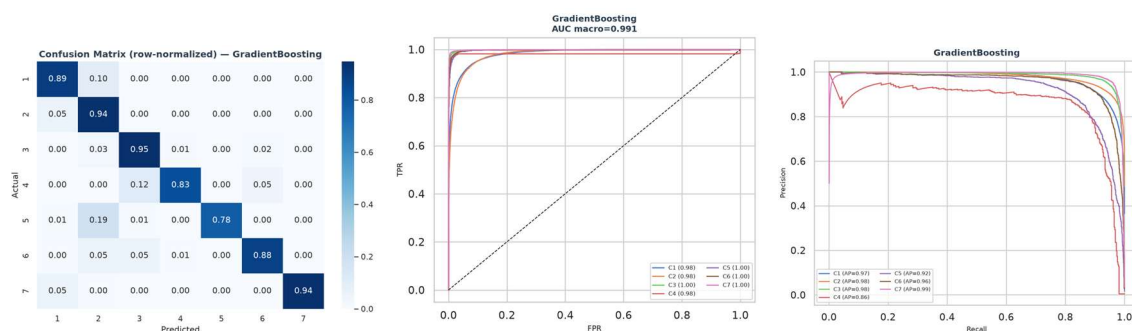
$F1\ CV = 0.891$ / $F1\ Test = 0.941$ / $F1\ Macro = 0.903$ / $Accuracy = 0.941$ / $AUC\text{-}macro = 0.944$



Il KNN con K=1 raggiunge $F1 = 0,941$ su tutto il dataset, poiché la copertura forestale contiene variabili geografiche continue e quindi ogni punto del test set ha probabilmente un vicino nel training set della stessa classe. Le confusioni principali sono tra 1 e 2, con le classi 5 e 6 che restano le più difficili da classificare. AUC-macro = 0,944 con curve a scalini.

4 — Gradient Boosting

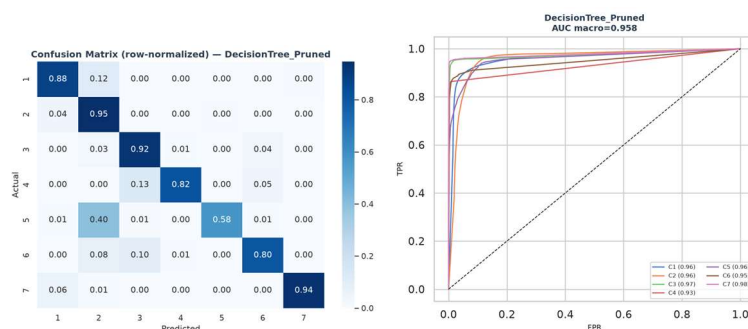
$F1\ CV = 0.886$ / $F1\ Test = 0.919$ / $F1\ Macro = 0.904$ / $Accuracy = 0.919$ / $AUC\text{-}macro = 0.991$



Il modello sbaglia principalmente sulle classi 1 e 2. Le classi rare 4 e 5 sono le più difficili anche in questo caso, ma rispetto al singolo albero potato il boosting riduce gli errori sulle classi minori. AUC-macro = 0,991 molto alto e le curve delle classi 1 e 2 mostrano una leggera flessione che indica la difficoltà di separare le due classi dominanti. Le classi 4–7 raggiungono invece $AUC \approx 1,00$.

5 — Decision Tree Pruned

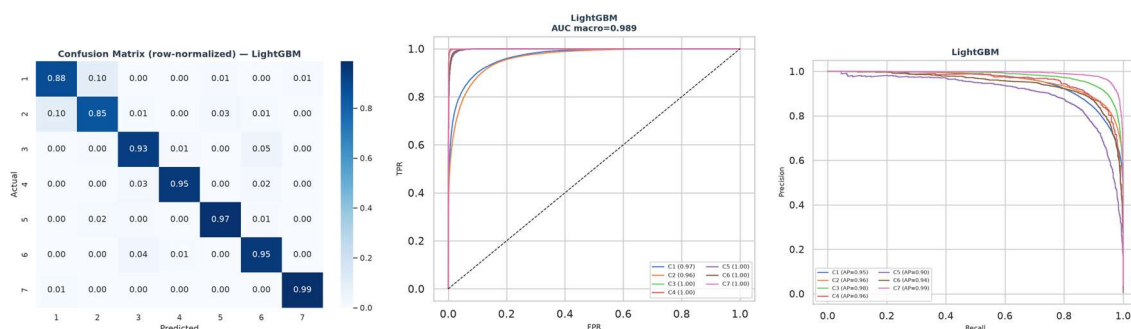
F1 CV = 0.855 | F1 Test = 0.908 | F1 Macro = 0.862 | Accuracy = 0.909 | AUC-macro = 0.958



Il singolo albero raggiunge $F1 = 0,908$ nonostante sia un modello semplice. La ricerca della profondità ottimale e di `ccp_alpha` per il cost-complexity pruning hanno permesso di evitare l'overfitting. Con molti campioni di training ogni foglia può contenere abbastanza osservazioni da costituire condizioni precise. Le confusioni sulle classi 1 e 2 sono più frequenti rispetto agli ensemble, la classe 5 con recall 0,58 è la più penalizzata. AUC-macro = 0,958 e le curve hanno una forma più spigolosa rispetto agli ensemble.

6 — LightGBM

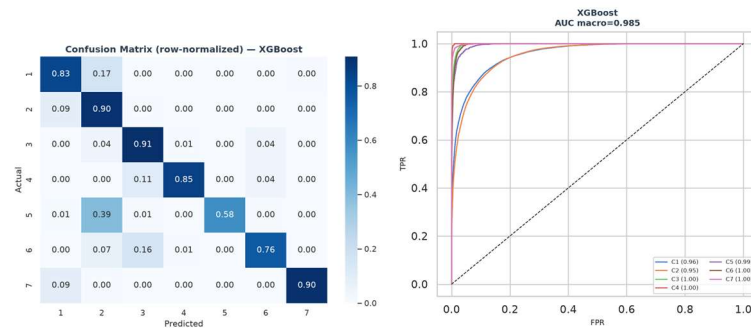
F1 CV = 0.870 | F1 Test = 0.877 | F1 Macro = 0.855 | Accuracy = 0.875 | AUC-macro = 0.989



LightGBM mostra performance più basse rispetto alle aspettative. La confusion matrix mostra che la classe 5 viene classificata in modo efficace (recall 0,97) ma con solo ~50% di precision, quindi il modello sovrastima la classe. La griglia degli iperparametri esplora anche la strategia di pesatura delle classi, e la configurazione selezionata permette di migliorare la recall sulle classi rare accettando falsi positivi da quelle dominanti, riducendo l' $F1$ pesato (0,8766), mentre l' $F1$ macro (0,8548) resta più alto. Questo modello è quarto per AUC-macro = 0,989 e dalle curve si osserva come faticosi nella predizione delle classi 1 e 2.

7 — XGBoost

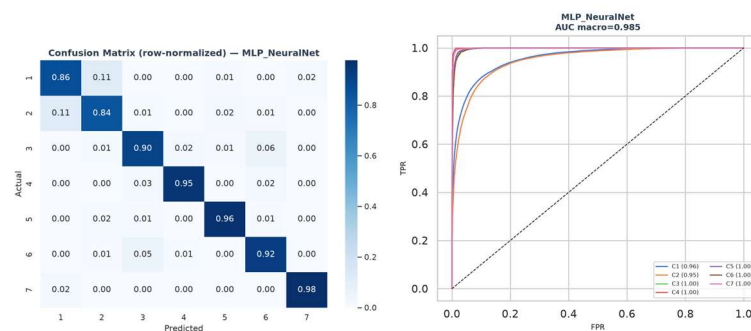
F1 CV = 0.848 | F1 Test = 0.862 | F1 Macro = 0.844 | Accuracy = 0.863 | AUC-macro = 0.985



XGBoost mostra la classica difficoltà sulle classi 1 e 2. La classe 5 è la più penalizzata coerentemente con la profondità massima esplorata dalla griglia (max_depth ≤ 7). AUC-macro = 0,985 con curve molto simili a quelle di LightGBM e le classi 1 e 2 difficili da separare.

8 — MLP NeuralNet

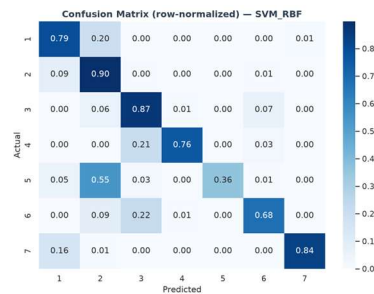
F1 CV = 0.821 | F1 Test = 0.861 | F1 Macro = 0.832 | Accuracy = 0.859 | AUC-macro = 0.985



L'MLP fa bene sulle classi rare (classe 4 con recall 0,95 e la 5 con recall 0,96) ma soffre di più sulle classi dominanti. La rete neurale ha appreso i confini non lineari per le classi rare meglio dei metodi ensemble, ma perdendo precisione globale. AUC-macro = 0,985 e le curve ROC sono quasi identiche a quelle di XGBoost.

9 — SVM RBF

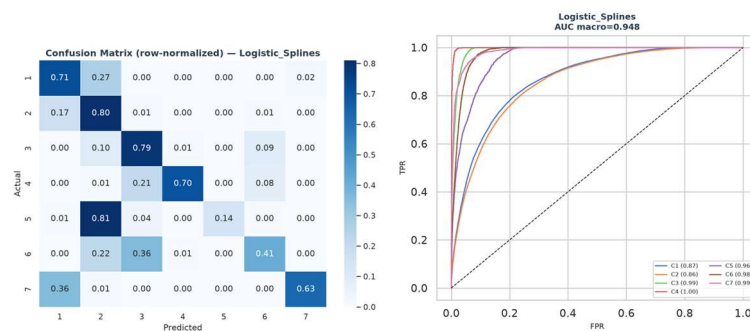
F1 CV = 0.833 | F1 Test = 0.838 | F1 Macro = 0.774 | Accuracy = 0.841 | AUC-macro: N/A



L'SVM è stato addestrato su un subset di 100.000 campioni per via della complessità $O(n^2)$ dell'algoritmo, ma generalizza comunque bene sulle 116.203 istanze di test. Il kernel RBF riesce a mappare i confini non lineari sicuramente meglio dei modelli lineari. La classe 5 è la più penalizzata (recall 0,36), poiché con il subset l'SVM ha visto pochi esempi. La classe 7 mostra molti falsi positivi verso la classe 1.

10 — Logistic Splines

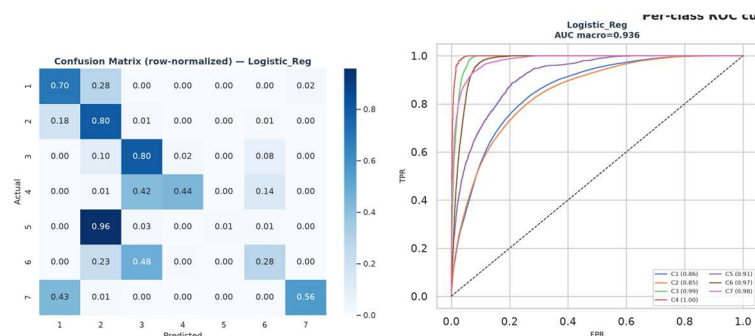
F1 CV = 0.738 | F1 Test = 0.736 | F1 Macro = 0.628 | Accuracy = 0.742 | AUC-macro = 0.948



Tra le regressioni logistiche, l'utilizzo di spline cubiche ha permesso di catturare relazioni non lineari. Gli errori tra le classi 1 e 2 restano alti e la classe 5 (recall 0,14) è quasi totalmente misclassificata. AUC-macro = 0,948 è molto alto, e la differenza tra AUC = 0,948 ed F1 = 0,736 mostra che vi è buona discriminazione ma soglie di classificazione imprecise.

11 — Logistic Reg

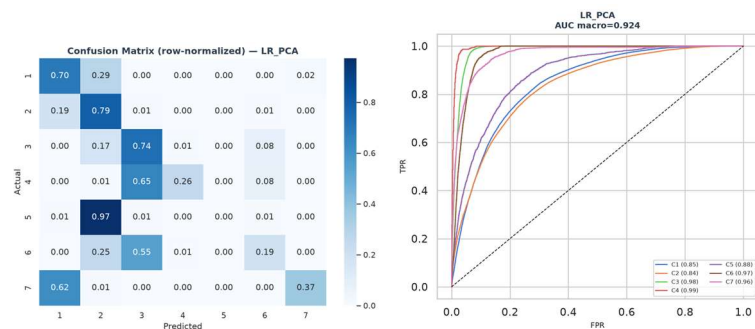
$F1\ CV = 0.718$ | $F1\ Test = 0.714$ | $F1\ Macro = 0.533$ | $Accuracy = 0.724$ | $AUC-macro = 0.936$



La regressione logistica con regolarizzazione L1 raggiunge $F1 = 0,714$. La classe 5 (recall 0,01) è praticamente ignorata e il modello predice quasi tutto come classe 1 o 2. La classe 7 viene spesso confusa con la 1. $AUC-macro = 0,936$, con le curve C1 e C2 molto distanti dalla diagonale.

12 — LR + PCA

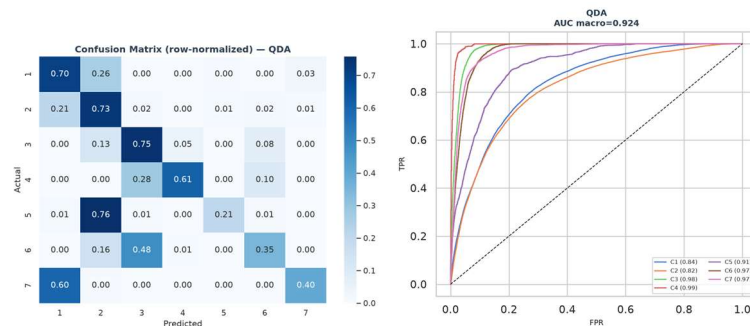
$F1\ CV = 0.697$ | $F1\ Test = 0.694$ | $F1\ Macro = 0.462$ | $Accuracy = 0.706$ | $AUC-macro = 0.924$



La variante con PCA ottiene $F1 = 0,694$, quindi il preprocessing con PCA porta una perdita di informazione. La riduzione a 12 componenti (95,3% di varianza) elimina l'informazione che distingue le classi rare. $AUC-macro = 0,924$ è il peggiore tra le regressioni lineari. La classe 4 raggiunge $AUC \approx 1,00$ nonostante la recall bassa.

13 — QDA

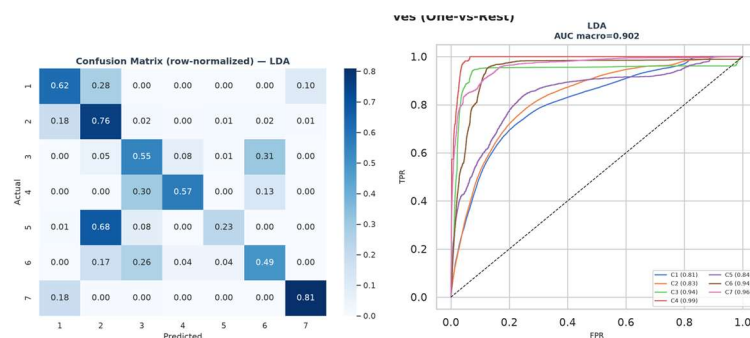
$F1\ CV = 0.687$ | $F1\ Test = 0.688$ | $F1\ Macro = 0.532$ | $Accuracy = 0.691$ | $AUC\text{-}macro = 0.924$



QDA mostra confini che migliorano leggermente rispetto a LDA. La classe 4 (recall 0,61) beneficia dei confini curvi. Le classi 5 (recall 0,21) e 6 (recall 0,35) restano problematiche. AUC-macro = 0,924 e le curve C1 e C2 mostrano la difficoltà del modello nel separare le due classi dominanti con metodi parametrici, mentre C4 (0,99) ha un AUC molto alto.

14 — LDA

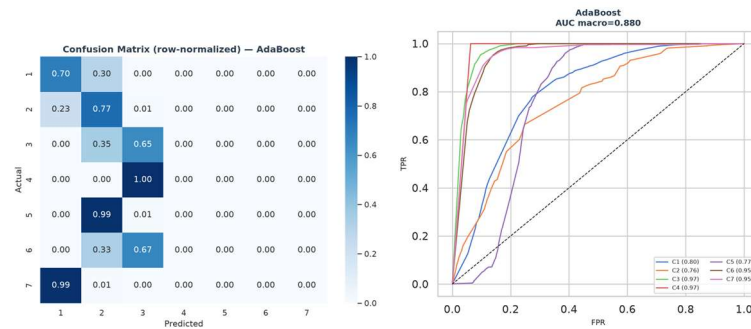
$F1\ CV = 0.684$ | $F1\ Test = 0.683$ | $F1\ Macro = 0.510$ | $Accuracy = 0.680$ | $AUC\text{-}macro = 0.902$



La matrice di LDA mostra molti falsi positivi dalla classe 1 verso la classe 7. LDA tende a confondere le classi agli estremi delle altitudini, un comportamento tipico della separazione lineare su distribuzioni multimodali. AUC-macro = 0,902, il peggiore tra i modelli parametrici, con le curve C1 e C2 molto basse.

15 — AdaBoost

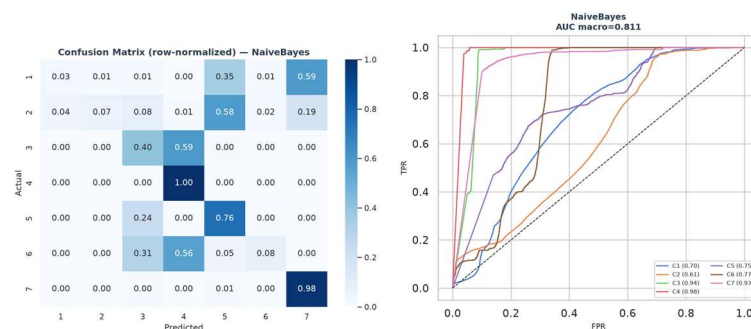
$F1\ CV = 0.643$ | $F1\ Test = 0.639$ | $F1\ Macro = 0.288$ | $Accuracy = 0.669$ | $AUC\text{-}macro = 0.880$



AdaBoost si concentra sugli esempi misclassificati e soffre molto in presenza di rumore nei confini di classe, quindi i campioni ambigui tra le classi 1 e 2 vengono continuamente rivisti e ripesati, rovinando le prestazioni. La confusion matrix mostra che le colonne 4, 5, 6 e 7 sono azzerate (recall = 0,00) quindi il modello predice esclusivamente le classi 1, 2 e 3. AUC-macro = 0,880 con curve molto basse.

16 — Naive Bayes

$F1\ CV = 0.112$ | $F1\ Test = 0.113$ | $F1\ Macro = 0.144$ | $Accuracy = 0.122$ | $AUC\text{-}macro = 0.811$



Il Naive Bayes è il modello meno adatto al problema, con $F1 = 0,113$. L'assunzione di indipendenza tra le feature non regge in un dataset geografico dove quota, pendenza e distanze idriche sono molto correlate. Nella matrice si osservano moltissimi falsi positivi sulla classe 5, e la classe 4 raggiunge recall = 1,00 ma precision = 0,07. AUC-macro = 0,811 è il peggiore tra tutti i modelli e le curve ROC sono molto instabili.

10 — Limiti

I risultati riportati valgono per il processo descritto nella Sezione 3, che ha dei limiti che vale la pena dichiarare esplicitamente. Per ciascun limite viene indicata anche la ragione per cui non è stato rimosso, alcune scelte sono deliberate e altre sono compromessi di costo computazionale.

Autocorrelazione spaziale. Le celle del dataset Covertypes sono geograficamente contigue e le osservazioni ne conservano l'ordine, ad esempio due righe consecutive differiscono in quota di 11,65 m in media, contro i 307,55 m di due righe estratte a caso, e il 95,1% delle righe consecutive appartiene alla stessa classe, contro il 37,7% atteso su un ordinamento casuale. Di conseguenza una divisione casuale dei dati in training e test set distribuisce celle adiacenti in entrambi i gruppi. Il sintomo più evidente è il KNN che raggiunge $F1 = 0,941$, il che indica che la maggior parte delle celle di test ha un vicino quasi identico nel training set. Uno split a blocchi contigui sarebbe il protocollo più severo ma adottarlo renderebbe i valori non confrontabili con la letteratura su Covertypes che si basa su split casuali.

Subset di tuning: La ricerca degli iperparametri è stata svolta su un subset stratificato di 100.000 osservazioni anziché sull'intero train-set e l'SVM è stato addestrato su quel subset per tutto il progetto. Gli ensemble di testa sono vicini alla saturazione sulla griglia di ricerca degli iperparametri, quindi estendere la ricerca all'intero train-set avrebbe moltiplicato il costo della pipeline per un ristretto guadagno di prestazioni sui modelli di fascia intermedia, senza modificare l'ordine delle prime posizioni né le conclusioni del confronto.

Unico split: L'intervallo di confidenza bootstrap misura il rumore di campionamento del test-set e non la varianza tra split diversi dei dati. Non è stata eseguita una cross-validation sull'intero dataset quindi l'intervallo va letto come la precisione di questa misura, non come la variabilità del metodo. Una validazione annidata moltiplica il calcolo per il numero di fold su una pipeline di 16 modelli, e risponderebbe a una domanda diversa da quella posta qui. Non cambierebbe inoltre l'esito del confronto poiché il divario tra il primo e il secondo classificato (0,9693 contro 0,9654) è circa il doppio dell'ampiezza dell'intervallo di confidenza (0,0020).

Probabilità quantizzate del KNN: Con $K = 1$ le probabilità predette assumono solo i valori 0 e 1 quindi le curve ROC e precision-recall del KNN sono composte da segmenti rettilinei. Il suo AUC-macro di 0,9440 non è confrontabile con quello dei modelli probabilistici perché sottostima un classificatore il cui $F1$ (0,9413) è il terzo in classifica.

Probabilità grezze: Tutte le analisi basate sulle probabilità usano gli output come vengono restituiti dagli stimatori. Non sono stati calcolati Brier score o soglie decisionali per classe poiché la calibrazione richiederebbe di ri-predire tutti i modelli e di rigenerare le figure che dipendono dalle probabilità. Ciò non cambierebbe la classifica perché l'ordinamento è deciso dall' $F1$ sulle predizioni finali e non dalle probabilità.

Gestione dello sbilanciamento parziale: I pesi di classe e SMOTE sono stati confrontati con l'addestramento sull'intero dataset, ma non sono stati testati l'undersampling delle classi dominanti, le soglie cost-sensitive o le focal loss. La conclusione che l'aumento dei dati abbia aiutato le classi rare più del ricampionamento è quindi limitata alle strategie testate. Ogni strategia aggiuntiva comporterebbe un nuovo ciclo di tuning e addestramento per i modelli coinvolti spostando lo scope di questo progetto.

Risultati dipendenti dall'ambiente: Rieseguendo la pipeline su una macchina e uno stack di librerie diversi i risultati si spostano sulla terza o quarta cifra decimale. Lo scostamento non tocca le classifiche né le conclusioni e `requirements.txt` fissa le versioni esatte con cui sono stati prodotti i log pubblicati. Inoltre il progetto produce un confronto e le sue evidenze, non un artefatto distribuibile poiché nessun modello viene serializzato e non esiste un'interfaccia di inferenza né un monitoraggio.

Nessuno di questi limiti mette in discussione il confronto in quanto tale poiché agiscono sul livello assoluto dei punteggi e non sull'ordinamento. L'unica asimmetria è quella già indicata sopra per il KNN.

11 — Conclusioni

Su questi dati esiste una gerarchia evidente tra le famiglie di modelli. Gli ensemble di alberi e i modelli basati sulle distanze occupano le prime posizioni con il Bagging Classifier in testa (F1 pesato 0,9693) e la Random Forest a seguire (0,9654), mentre i lineari e i parametrici non superano lo 0,74. La ragione sta nella natura del dataset con quota, pendenza e distanze che generano confini decisionali non lineari.

Il risultato più rilevante è che su un dataset in cui la classe maggiore vale il 48,76% delle osservazioni e la minore lo 0,47%, l'accuracy e l'AUC possono apparire solide mentre il modello ha smesso di predire quattro classi su sette. AdaBoost raggiunge accuracy 0,6688 e AUC-macro 0,8804 con recall 0,00 sulle classi 4, 5, 6 e 7. Due metriche aggregate lo descrivono come un modello funzionante ma la recall per classe mostra che ne ha abbandonate quattro. Per questo motivo ogni tabella del report affianca l'F1 macro a quello pesato e ogni modello viene riportato classe per classe.

Sulle classi rare l'aumento dei dati è stato l'intervento più efficace, nella run sul subset ristretto le classi 4 e 5 avevano recall 0,61 e 0,43, mentre sull'intero dataset il Bagging Classifier le porta entrambe a 0,88.

L'apporto più concreto di questo progetto è il metodo e la pipeline con cui la classifica è stata costruita, con sedici modelli addestrati sullo stesso split, test-set isolato e risultati letti per classe.